C:\hpc-23-feb-2026\.\17\eg144.c

```
// Example : Problem statement related to communication between threads
// issue : synchronization is missing
#include<stdio.h>
#include<pthread.h>
struct mdm
{
int num;
};
void set_num(struct mdm *m,int num)
{
m->num=num;
printf("Produced : %d\n",m->num);
}
int get_num(struct mdm *m)
{
printf("Consumed : %d\n",m->num);
return m->num;
}
void * produce(void *v)
{
struct mdm *m;
int i;
m=(struct mdm *)v;
for(i=201;i<=250;++i)
{
set_num(m,i);
}
return NULL;
}
void * consume(void *v)
{
struct mdm *m;
int j,k;
m=(struct mdm *)v;
for(j=1;j<=50;++j)
{
k=get_num(m);
}
return NULL;
}
int main()
{
struct mdm m;
pthread_t pid,cid;
int i1,i2;
i1=pthread_create(&pid,NULL,produce,(void *)&m);
i2=pthread_create(&cid,NULL,consume,(void *)&m);
pthread_join(pid,NULL);
```

```
pthread_join(cid,NULL);
return 0;
}
```

C:\hpc-23-feb-2026\.\17\eg145.c

```
// Example : Problem statement related to communication between threads
// issue : Wrong implementation, wait / notify is required
#include<stdio.h>
#include<pthread.h>
struct mdm
{
int num;
pthread_mutex_t *mtx; // it should not be here, there should be a separate struct
};
void set_num(struct mdm *m,int num)
{
m->num=num;
printf("Produced : %d\n",m->num);
}
int get_num(struct mdm *m)
{
printf("Consumed : %d\n",m->num);
return m->num;
}
void * produce(void *v)
{
struct mdm *m;
int i;
m=(struct mdm *)v;
for(i=201;i<=250;++i)
{
pthread_mutex_lock(m->mtx);
set_num(m,i); // production in action
pthread_mutex_unlock(m->mtx);
}
return NULL;
}
void * consume(void *v)
{
struct mdm *m;
int j,k;
m=(struct mdm *)v;
for(j=1;j<=50;++j)
{
pthread_mutex_lock(m->mtx);
k=get_num(m); // consumption in action
pthread_mutex_unlock(m->mtx);
}
return NULL;
}
int main()
{
struct mdm m;
```

```
pthread_mutex_t mtx;
pthread_t pid,cid;
int i1,i2;
pthread_mutex_init(&mtx,NULL);
m.mtx=&mtx;
i1=pthread_create(&pid,NULL,produce,(void *)&m);
i2=pthread_create(&cid,NULL,consume,(void *)&m);
pthread_join(pid,NULL);
pthread_join(cid,NULL);
pthread_mutex_destroy(&mtx);
return 0;
}
```

Thinking Machines Classroom Codes

C:\hpc-23-feb-2026\.\17\eg146.c

```
// this example has a serious BUG
#include<stdio.h>
#include<pthread.h>
struct mdm
{
int num;
pthread_mutex_t *mtx; // it should not be here, there should be a separate struct
pthread_cond_t *producer_condition_var; // it should not be here, there should be a separate struct
pthread_cond_t *consumer_condition_var; // it should not be here, there should be a separate struct
bool produce; // for should produce or not
};
void set_num(struct mdm *m,int num)
{
m->num=num;
printf("Produced : %d\n",m->num);
}
int get_num(struct mdm *m)
{
printf("Consumed : %d\n",m->num);
return m->num;
}
void * produce(void *v)
{
struct mdm *m;
int i;
m=(struct mdm *)v;
pthread_mutex_lock(m->mtx);
for(i=201;i<=250;++i)
{
if(m->produce==false) pthread_cond_wait(m->producer_condition_var,m->mtx);
set_num(m,i); // production in action
m->produce=false;
pthread_cond_signal(m->consumer_condition_var);
}
pthread_mutex_unlock(m->mtx);
return NULL;
}
void * consume(void *v)
{
struct mdm *m;
int j,k;
m=(struct mdm *)v;
pthread_mutex_lock(m->mtx);
for(j=1;j<=50;++j)
{
if(m->produce==true) pthread_cond_wait(m->consumer_condition_var,m->mtx);
k=get_num(m); // consumption in action
m->produce=true;
```

```
pthread_cond_signal(m->producer_condition_var);
}
pthread_mutex_unlock(m->mtx);
return NULL;
}
int main()
{
struct mdm m;
pthread_mutex_t mtx;
pthread_t pid,cid;
pthread_cond_t producer_condition_var;
pthread_cond_t consumer_condition_var;
int i1,i2;
pthread_mutex_init(&mtx,NULL);
pthread_cond_init(&producer_condition_var,NULL);
pthread_cond_init(&consumer_condition_var,NULL);
m.mtx=&mtx;
m.producer_condition_var=&producer_condition_var;
m.consumer_condition_var=&consumer_condition_var;
m.produce=true;
i1=pthread_create(&pid,NULL,produce,(void *)&m);
i2=pthread_create(&cid,NULL,consume,(void *)&m);
pthread_join(pid,NULL);
pthread_join(cid,NULL);
pthread_mutex_destroy(&mtx);
pthread_cond_destroy(&producer_condition_var);
pthread_cond_destroy(&consumer_condition_var);
return 0;
}
```

C:\hpc-23-feb-2026\.\17\eg147.c

```
// Bug of previous example resolved
#include<stdio.h>
#include<pthread.h>
struct mdm
{
int num;
pthread_mutex_t *mtx; // it should not be here, there should be a separate struct
pthread_cond_t *producer_condition_var; // it should not be here, there should be a separate struct
pthread_cond_t *consumer_condition_var; // it should not be here, there should be a separate struct
bool produce; // for should produce or not
};
void set_num(struct mdm *m,int num)
{
m->num=num;
printf("Produced : %d\n",m->num);
}
int get_num(struct mdm *m)
{
printf("Consumed : %d\n",m->num);
return m->num;
}
void * produce(void *v)
{
struct mdm *m;
int i;
m=(struct mdm *)v;
pthread_mutex_lock(m->mtx);
for(i=201;i<=250;++i)
{
while(m->produce==false) pthread_cond_wait(m->producer_condition_var,m->mtx);
set_num(m,i); // production in action
m->produce=false;
pthread_cond_signal(m->consumer_condition_var);
}
pthread_mutex_unlock(m->mtx);
return NULL;
}
void * consume(void *v)
{
struct mdm *m;
int j,k;
m=(struct mdm *)v;
pthread_mutex_lock(m->mtx);
for(j=1;j<=50;++j)
{
while(m->produce==true) pthread_cond_wait(m->consumer_condition_var,m->mtx);
k=get_num(m); // consumption in action
m->produce=true;
```

```
pthread_cond_signal(m->producer_condition_var);
}
pthread_mutex_unlock(m->mtx);
return NULL;
}
int main()
{
struct mdm m;
pthread_mutex_t mtx;
pthread_t pid,cid;
pthread_cond_t producer_condition_var;
pthread_cond_t consumer_condition_var;
int i1,i2;
pthread_mutex_init(&mtx,NULL);
pthread_cond_init(&producer_condition_var,NULL);
pthread_cond_init(&consumer_condition_var,NULL);
m.mtx=&mtx;
m.producer_condition_var=&producer_condition_var;
m.consumer_condition_var=&consumer_condition_var;
m.produce=true;
i1=pthread_create(&pid,NULL,produce,(void *)&m);
i2=pthread_create(&cid,NULL,consume,(void *)&m);
pthread_join(pid,NULL);
pthread_join(cid,NULL);
pthread_mutex_destroy(&mtx);
pthread_cond_destroy(&producer_condition_var);
pthread_cond_destroy(&consumer_condition_var);
return 0;
}
```